DESIGN DOCUMENT

ClearFrame

Automated ad compliance and brand safety for creator video ads, built on TwelveLabs

Use case: a global makeup brand promoting creator beauty videos as paid ads
Decision output: APPROVE / REVIEW / BLOCK, with an auto-generated description and timestamped evidence

---

ClearFrame is an automated gate that decides, for every creator video an advertiser wants to promote, whether it can run as a paid ad. It returns an APPROVE, REVIEW or BLOCK verdict, a two to five sentence description of the video, and timestamped evidence for every reason behind the call. TwelveLabs handles video perception across visuals, speech and on-screen text. A deterministic policy engine makes the verdict. This document explains how decisions are made, why the outputs are trustworthy, and how the system scales in a real ads pipeline.

1 The problem, stated as the buyer sees it

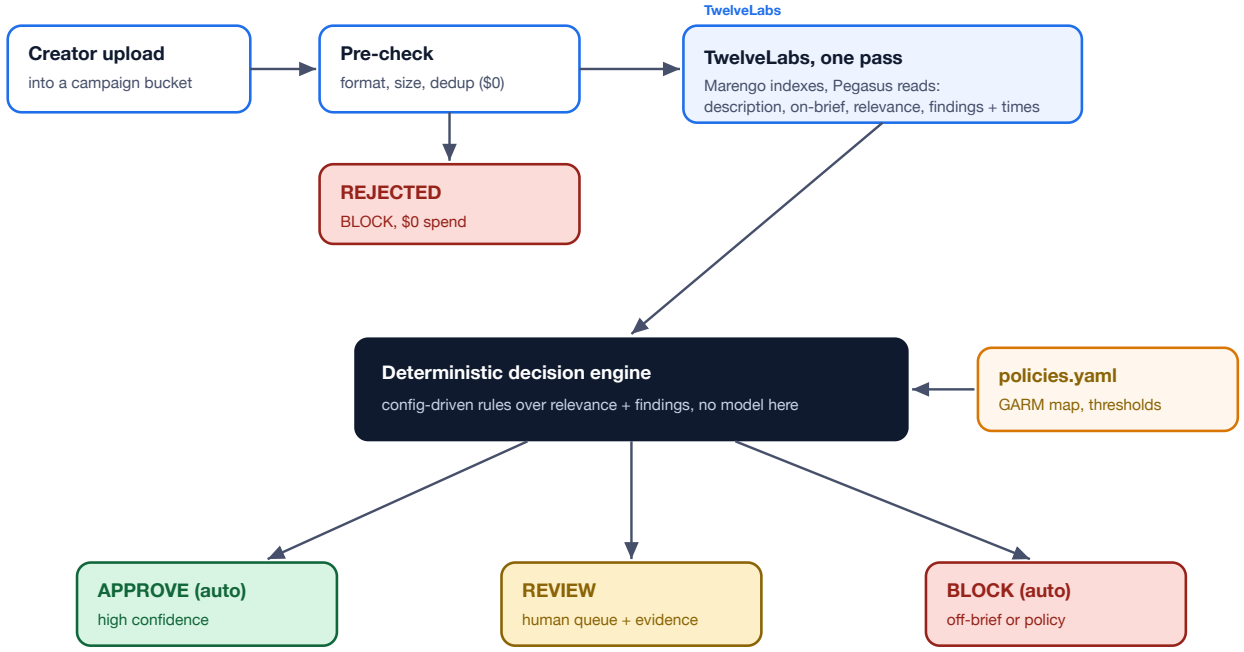
The platform's revenue depends on video advertising, and advertisers increasingly run creator-generated content as the ad creative itself. Before a beauty brand lets a creator video run as a paid ad, it expects brand safety, compliance and explainability guarantees. Today that check is manual: inconsistent between reviewers, slow, and impossible to run at the volume of submissions a large platform receives.

The hard parts are specific. Risky content can appear for only a second, in a visual, a spoken phrase, or on-screen text. Subtle violations such as unsafe product application or a medical claim dressed up as a beauty tip are easy to miss. Some submissions are off-brief: creator content that barely features the product, or unrelated clickbait. And when an ad is blocked, the advertiser and the reviewer both need to know exactly why.

> **The design thesis**
> The model perceives. The engine decides. TwelveLabs understands what happens in the video. A separate, deterministic policy engine turns that understanding into the verdict. No language model makes the final compliance call on its own. In an ads system a wrong automated block on a large advertiser is a revenue and trust incident, so the decision stays in inspectable, testable, per-advertiser-tunable rules.

2 System architecture

The pipeline runs three stages behind an async ingestion front end. A local, deterministic pre-check runs first, format, size, duration, and a content-hash dedup, so a malformed or duplicate submission never spends a TwelveLabs call. What passes goes to TwelveLabs for one perception pass, then a deterministic engine decides.



One TwelveLabs pass produces everything perception-related in a single structured response: the description, the on-brief judgment and relevance score, and per-policy findings with timestamps. The deterministic engine downstream never calls a model; it compares those findings against the campaign's own thresholds.

Why one pass instead of two, and why not a single "is this safe" prompt

A single "watch this and tell me if it is safe" call is a demo, not a system: it returns prose you cannot audit and a decision you cannot unit-test. The fix is not more model calls, it is separating perception from decision. One schema-constrained Pegasus call reads visuals, speech and on-screen text and returns structured findings, on-brief judgment, and relevance together; a separate, deterministic engine turns those findings into the verdict. No language model makes the final compliance call. An earlier version of this design used a second TwelveLabs pass (Marengo search) to independently corroborate each finding. That pass was dropped: Marengo's `/search` stopped returning a numeric score in the API version this was built against, so it could no longer corroborate by magnitude. Marengo still indexes every video and remains available for reviewer-driven search, it is simply outside the automated decision path now. Relevance and evidence both come from the single Pegasus pass instead.

3 Policy taxonomy

Policies live in configuration, mapped to the GARM Brand Safety Floor and Suitability Framework. Each policy declares whether it is a GARM floor category, its suitability tier, and the severity at which it blocks or routes to review. Adding a category or changing a tolerance for a new advertiser is a config edit, not a code change.

Policy	GARM	Tier	Blocks at	Reviews at
Hate, harassment or discrimination	Floor	High	medium	low
Drug use or illegal behaviour	Floor	High	medium	low
Profanity or explicit language	Suitability	Medium	high	low
Unsafe or misleading product usage	Suitability	High	medium	low
Medical or unsubstantiated cosmetic claims	Suitability	High	medium	low
Campaign relevance (on-brief)	Gate	n/a	off-brief	n/a

> **On GARM, stated honestly**
> GARM the organisation was disbanded by the World Federation of Advertisers in August 2024. Its Brand Safety Floor and Suitability Framework remain the de facto industry standard, embedded in the major verification vendors. ClearFrame adopts the framework, not the defunct body. A careful reader will know this, so the doc states it plainly.

Beauty-specific compliance: the cosmetic-versus-drug line

The most important regulatory boundary for makeup ads is the line the FDA draws between a cosmetic and a drug. A product claim crosses into drug territory when it claims to treat, cure, or affect the structure or function of the body, for example "cures acne" or "treats rosacea". Those claims are not permitted in a cosmetic ad without drug approval, so ClearFrame treats them as blocking. Claims that stay in beautify, cleanse and moisturise territory are acceptable. This boundary is nuanced and a production deployment should involve counsel to maintain the claims taxonomy.

4 How the decision is made

Severity runs none, low, medium, high, set by Pegasus per policy in the single analyze call. The decision engine compares each finding's severity against that campaign's own `block_at` and `review_at` thresholds. Verdict precedence is BLOCK over REVIEW over APPROVE.

1. Off-brief (relevance below threshold and Pegasus judged it off-brief) results in BLOCK; the creative is not eligible to run.
2. Any policy at or above its block severity results in BLOCK. GARM floor categories (hate/harassment, drugs/illegal) go through this identical check, they are not a separate step.
3. Any policy at or above its review severity routes to human REVIEW.

4. Otherwise, APPROVE.

Confidence and human-in-the-loop. A clean APPROVE (no findings at all) is high confidence; a high-severity finding driving the verdict is high, anything less is medium. Any automated APPROVE or BLOCK below the configured minimum confidence is downgraded to REVIEW so a human looks at it, the one exception is a GARM-floor BLOCK, which stays BLOCK regardless of confidence. That exception, not a corroboration requirement, is what keeps a floor violation (hate speech, drug use) from silently routing to a human queue on a low-confidence read.

Worked examples from the prototype

Creative	Verdict	Why
Clean product tutorial	APPROVE	On-brief, relevance 0.80, every policy clean. High confidence.
Eye-area misuse plus medical claim	BLOCK	Unsafe usage and a drug claim, both at high severity, each with its own timestamp from the same Pegasus pass (34.0s-41.5s, 58.0s).
One mild swear word, otherwise on-brief	REVIEW	Profanity at low severity, below the block threshold but above review. Routed to a human with the clip at 47.0s.
Off-brief lifestyle clickbait	BLOCK	Relevance 0.20, product only appears incidentally in the background. The same Pegasus call that read the video judged it off-brief; no separate pass, no extra cost.

5 Why the outputs are trustworthy

- **The model perceives, it does not decide.** Pegasus's job ends at producing findings. Turning those findings into APPROVE, REVIEW or BLOCK is a separate step in plain code, no model in that path at all.
- **Deterministic, testable verdicts.** Because the decision is rules over signals, it is unit-testable. The prototype ships tests proving that a given set of signals always yields the same verdict, and a labeled evaluation set (16 cases today, a real regression gate, not a claim of finished coverage) proves the engine stays correct as policies change. You cannot write either test against "ask the model to decide".
- **Timestamped evidence per reason.** Every blocking or review reason carries a start and end timestamp grounded in the same Pegasus finding, so a reviewer or advertiser can watch the exact moment.
- **Humans on uncertainty.** Low-confidence calls and the entire REVIEW tier route to human moderators by design, with evidence pre-attached, and a reviewer can override any verdict.
- **Auditable by construction.** Each decision traces to a rule, a threshold version, and a piece of evidence, which is what an advertiser dispute six weeks later actually requires.

6 Scaling in a real ads system

- **Async indexing.** Submit to the indexing endpoint and resume on the `index.task.ready` webhook (HMAC-signed and verified). Never block a worker on a long indexing job.
- **Index once, reuse for reviewer search.** The Marengo index built at ingest time backs ad-hoc reviewer-driven search later, it is not re-created per query.
- **A real job queue, not yet built.** The prototype runs each review on an in-process background thread, which is exactly the shape a queue consumer should take, but it does not survive a process restart. That was found the hard way during this build: a server restart mid-review left it stuck showing "processing" forever, since fixed to fail loudly instead of hanging, but the actual fix, a durable queue in front of a worker pool, is the honest next step before this runs unattended.
- **Per-advertiser tenancy.** Each advertiser gets its own policy config and thresholds. Onboarding scales through configuration, not engineering. There is currently no auth separating one campaign's data from another's, that has to land before multi-tenant is real.
- **Cost control as a design input.** The pre-check, not a relevance gate, is what actually protects spend: it is a local, deterministic check that rejects malformed files and duplicates before a single TwelveLabs call is made. Relevance itself is judged by the same Pegasus call that reads the video, so an off-brief creative still costs one analyze call; there is no cheaper pre-filter ahead of it in the current design.

7 Build versus buy: why TwelveLabs over a DIY stack

This is the question an evaluator will press hardest, so it is answered directly.

Dimension	DIY stack (Whisper + a vision model + an LLM)	TwelveLabs

Temporal understanding	General vision models sample roughly one frame per second and treat frames as discrete snapshots, losing temporal continuity. A one-second violation between sampled frames can be missed.	Video-native models reason over time and space directly.
What you own and maintain	ASR, frame sampling, vision inference, prompt stitching, a vector database, evaluation and ongoing model upkeep.	One platform with search and generative analysis; the team focuses on policy logic, not plumbing.
Reviewer search after the fact	Re-running detection or search means re-paying for inference each time.	The Marengo index built at ingest time is reused for any later reviewer-driven search, no re-indexing per query.
Evidence	You build timestamp extraction yourself.	One structured Pegasus call returns per-finding timestamps natively, no separate extraction step.

Against hyperscaler moderation (AWS Rekognition, Google Video Intelligence, Azure Video Indexer): those excel at fixed-taxonomy primitives such as NSFW and violence with deep cloud-native integration, but offer narrower categories and less open-ended reasoning. TwelveLabs differentiates on semantic search plus generative reasoning plus custom natural-language policy without retraining. If a buyer needs only binary NSFW detection at massive scale on AWS, a Rekognition-first design may win on cost; for nuanced, explainable, custom brand suitability with semantic evidence, TwelveLabs is the stronger fit. The honest answer is a hybrid: cheap pre-filter, deep analysis where it matters, humans on the edges.

8 Honest limitations and what to verify

- The generative model can hallucinate, which is precisely why the decision is deterministic and REVIEW routes to humans.
- The eval harness proves the decision engine is correct given a fixed set of Pegasus outputs. It does not prove Pegasus itself correctly sees a violation in a real video; that needs labeled real footage and is a separate, harder problem. One representative live run is documented separately, it is a single example, not an accuracy benchmark.
- Relevance and on-brief both come from the same Pegasus call that reads the video. That is a self-reported judgment, not an independently measured signal the way an external search score would be, and it is possible for the two fields to disagree in edge cases.
- Indexing adds latency on long videos; the pipeline is built async to absorb it.
- The build runs on Pegasus 1.2. Pegasus 1.5 is available with a larger context window and no pre-indexing requirement for general analysis; migrating is a real code change, not done here.
- The cosmetic-versus-drug claim boundary is nuanced; a real deployment should involve counsel.
- The TwelveLabs API surface moves quickly. Model names, endpoints and limits should be re-verified against live documentation before relying on them; several endpoints have been renamed or retired in recent releases.

> **Integration reference used in the build**

> Marengo for multimodal indexing and reviewer-driven search; Pegasus for generative analysis with structured JSON output, description, on-brief judgment, relevance score, and per-policy findings with timestamps, all from one call. REST endpoints used: indexing tasks, and `/analyze` with a JSON schema for the structured decision output. `/search` is available for reviewer-driven exploratory search but is not in the automated decision path; its response in this API version no longer includes a numeric score. The prototype talks to the documented REST surface directly so it controls retries, timeouts and error handling, with a fixture-backed mock layer so the full pipeline runs in CI and in a live demo without a key. Exact model versions and endpoint paths should be confirmed against the live TwelveLabs API before deploying.
